

DEVELOPER PORTFOLIO

이우람

APPROACH

문제를 재현하고 원인을 좁혀 해결합니다
현상을 바로 고치기보다 요청 흐름·데이터 경계·실행 환경을 분리해 재현 조건을 만들고
두 프로젝트에서 동시성·트랜잭션·테스트 실행 문제를 구조적으로 해결했습니다.

TOOLS

Spring Boot

Testcontainers

PostgreSQL

MySQL

Redis

Kafka

Next.js

Docker

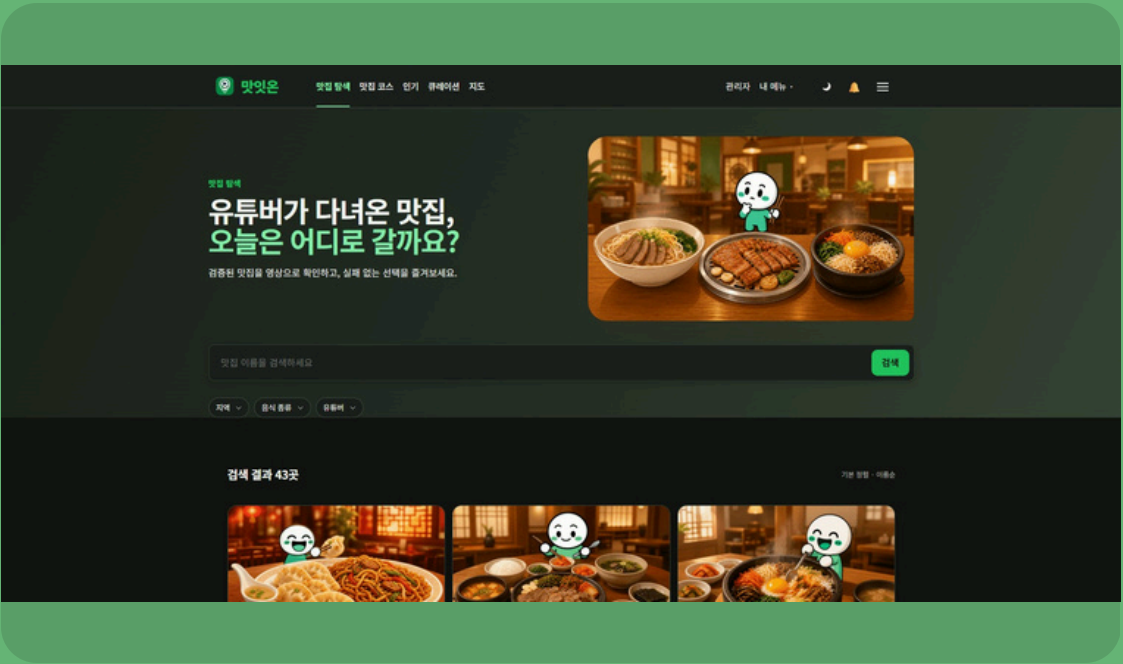
EVIDENCE

동시성 재현 같은 좌석 10개 동시 요청과 결제 재요청을 테스트로 재현했습니다. 결제 멍등성·Redis 분산락·AI 등록 트랜잭션 경계 설계	경계로 실패 격리 AI 후보 검증과 DB 저장 경계를 나눠 실패가 전체 등록으로 번지지 않게 했습니다. AI 후보 검증을 맛집별 등록 단위로 분리해 실패 범위 격리	실행 구조 개선 1,400개 테스트를 8분 20초+에서 4분 32초로 단축했습니다. Testcontainers 기동 52회 → 2회, 전체 테스트 8분 20초+ → 4분 32초
--	---	--

맛잇온

유튜버 맛집을
지역·음식 종류·유튜버별로 탐색하는 서비스

유튜버가 방문한 맛집을 지역·음식 종류·유튜버별로 탐색하는 서비스입니다.
백엔드·인프라 개발과 일부 프론트엔드 구현, 프로젝트 진행 관리를 맡았습니다.



DATE	TEAM	ROLE
2026.07 ~ 2026.08	백엔드 4명	백엔드 / 인프라 개발 · 프론트엔드 일부 구현 · 프로젝트 진행 관리

Next.js 프론트엔드

Nginx

Spring Boot API 서버

PostgreSQL

Redis

GitHub Actions

Docker / Terraform / AWS

GitHub 레포 ↗

INSIGHT

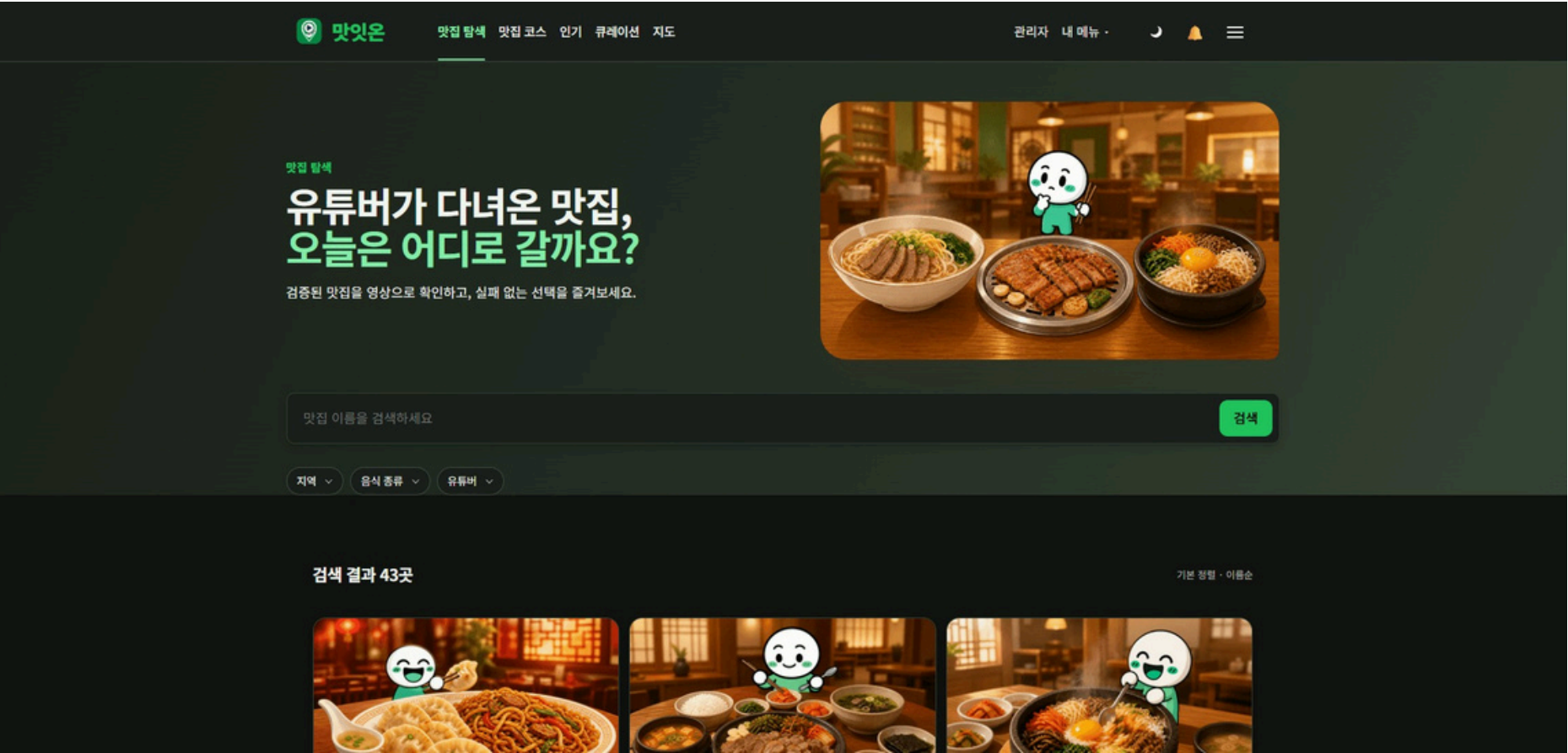
외부 검증과 DB 저장을 분리하고
최종 반영 시 상태를 다시 확인해 중복 등록을 막았습니다.

AI 결과는 맛집별 등록 단위로 나눠
한 건의 실패가 전체 등록을 막지 않도록 했습니다.

맛잇온

유튜버가 방문한 맛집을 지역·음식 종류·유튜버별로 탐색할 수 있는 맛집 정보 서비스입니다.

검색·지도 탐색과 백엔드·인프라를 직접 구축해 실제 서비스로 운영했습니다.



DATE

2026.07 – 2026.08

TEAM

백엔드 4명

ROLE

백엔드·인프라 / 프론트 일부

Next.js 프론트엔드
지도·맛집 탐색 화면과 인증 흐름 제공

Nginx
공개 경로 라우팅과 클라이언트 IP 전달

Spring Boot API 서버
맛집 검색·인증·제보·신고 처리

PostgreSQL
맛집·지역·음식 종류 저장

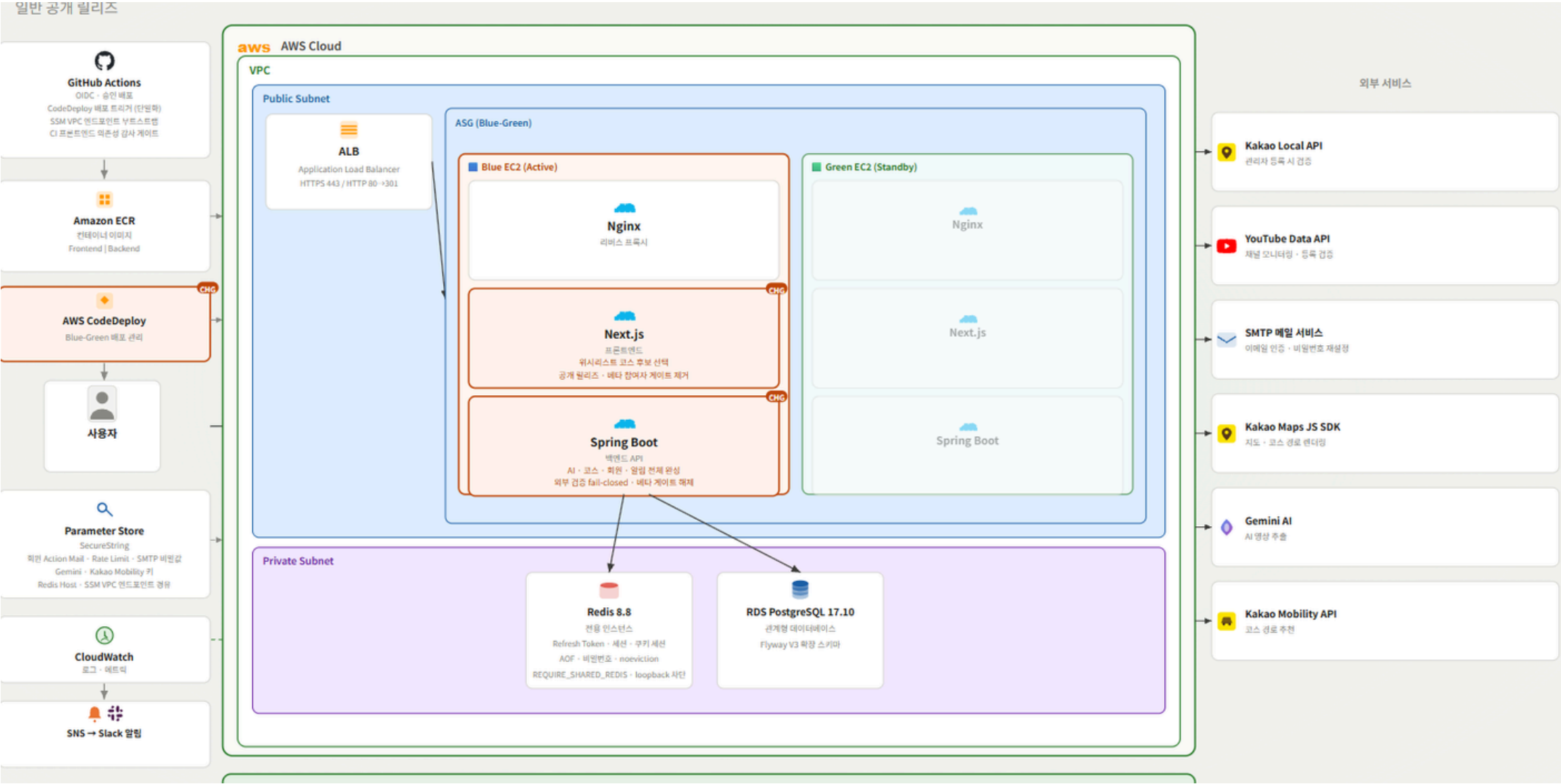
Redis
로그인 Rate Limit·캐시·요청 상태 관리

Docker / Terraform / AWS
애플리케이션 실행·운영 환경 구성

GitHub Actions
빌드·테스트·배포 자동화

요청을 검증하고 저장 경계를 분리한 구조

외부 검증은 요청 흐름에서 처리하고, DB 저장은 필요한 구간만 트랜잭션으로 묶었습니다.
핵심 상태를 먼저 확인한 뒤, 테스트와 후속 처리를 별도 경계로 분리했습니다.



AI 자동 등록의 중복과 부분 저장을 경계 설계로 막았습니다

CONTEXT

동일 맛집에 요청이 동시에 들어오면
중복 등록이나 부분 저장이 생길 수 있었습니다.

DECISION



외부 API 검증과 DB 저장을 분리하고,
맛집 단위의 저장 경계를 하나의 작업으로 묶었습니다.
최종 반영 전 상태도 다시 확인했습니다.

검증 기준

동시 요청과 저장 실패를 경계 단위로 재현했습니다.

동일 맛집 동시 요청

- 외부 검증과 DB 작업 분리
- 최종 반영 전 상태 재확인
- 신규 데이터만 하나의 저장 단위로 처리

PROOF

한 요청만 최종 반영하고,
실패 요청은 생성한 데이터만 정리해 재시도할 수 있게 했습니다.

테스트 실행 환경을 재사용해 1,400개 테스트를 4분 32초로 단축했습니다

CONTEXT

컨테이너 반복 기동과 공유 상태 오염으로
테스트가 느리고 간헐적으로 실패했습니다.

DECISION



컨테이너와 Spring 환경을 테스트 프로세스 동안 재사용하고,
DB·Redis 정리 범위를 기능 단위로 좁혔습니다.

PROOF

1,400개 테스트를 유지하면서
8분 20초+ → 4분 32초,
컨테이너 기동 52회 → 2회로 줄였습니다.

검증 결과

실행 시간·기동 횟수·공유 상태 격리를 함께 확인했습니다.

1,400 tests

실행 시간 8m 20s+ → 4m 32s

컨테이너 기동 52회 → 2회

상태 격리

PostgreSQL FK 순서 정리

Redis 키 접두사만 삭제

실행 구조를 개선한 검증 가능한 설계 판단

DECISION 01

저장 직전 상태 재검증
동시 요청에서는 최초 조회 결과를 그대로 믿지 않고, 저장 직전 상태를 다시 확인했습니다.

DECISION 02

외부 I/O와 DB 트랜잭션 분리
외부 검증은 트랜잭션 밖에서 처리하고, 저장이 필요한 구간만 작업 단위로 관리했습니다.

DECISION 03

공통 환경 재사용
Testcontainers와 Spring 실행 환경을 재사용해 반복 기동 비용을 줄이면서도 테스트 수를 유지했습니다.

DECISION 04

실행 비용 개선
Testcontainers 기동 52회 → 2회
전체 테스트 8분 20초+ → 4분 32초

동시성·먹등성으로 좌석·결제 정합성 지키기

대기열 진입부터 좌석 예약, 결제까지 이어지는
티케팅 서비스를 백엔드 중심으로 구현했습니다.

동일 좌석 동시 요청과 결제 재요청을 테스트로 재현하고,
Redis 분산락과 reservationId 먹등성으로 중복·정합성 문제를 해결했습니다.

DATE	TEAM	ROLE
2025.11 ~ 2026.05	1인	백엔드 개발

API 서버(Spring Boot)

MySQL

Redis

Kafka

GitHub 레포 ↗



INSIGHT

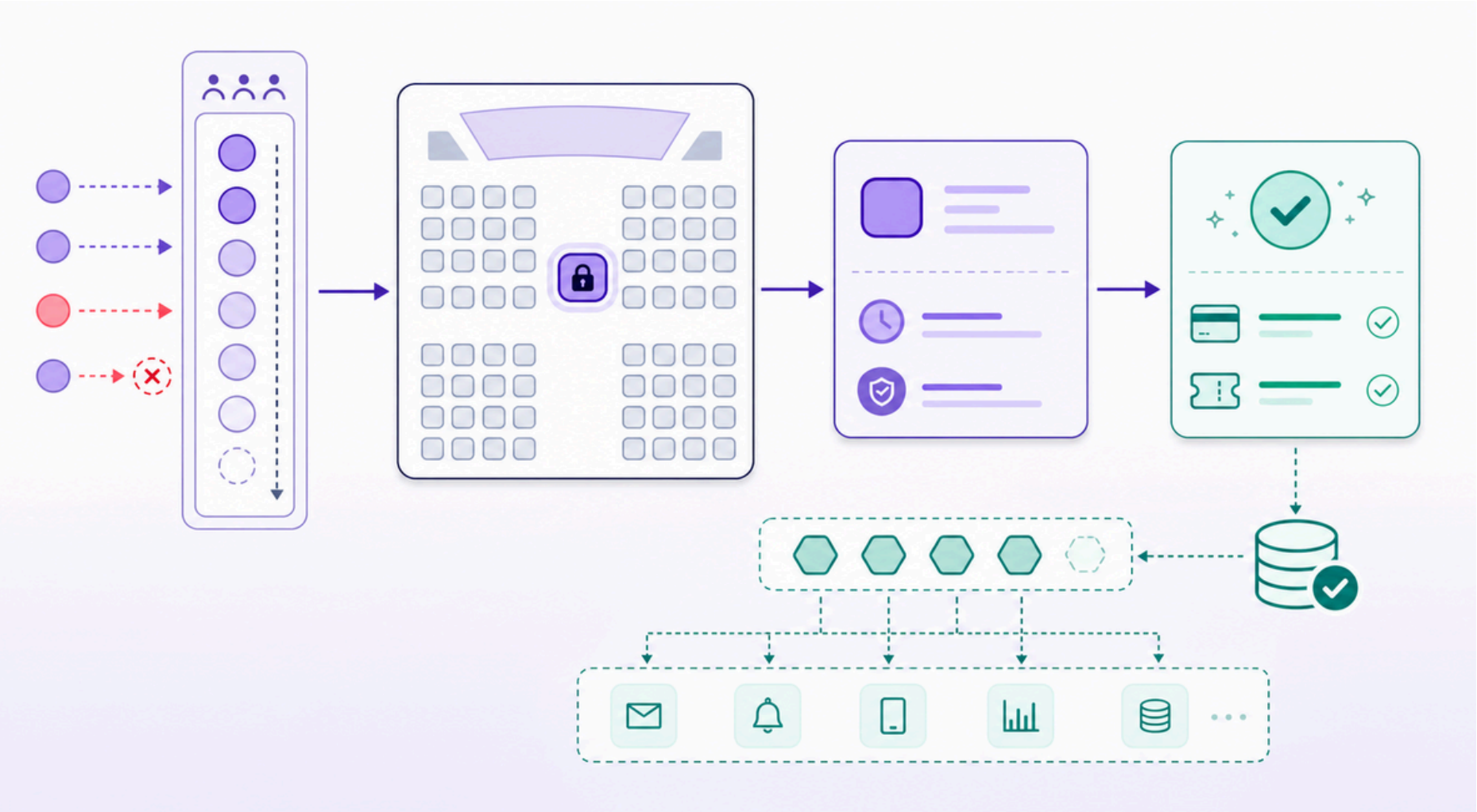
좌석 ID 기준으로 경쟁을 직렬화하고
reservationId 기준으로 결제 재요청을
같은 결과로 수렴시켰습니다.

동시 요청과 재시도를 통합 테스트로 검증했습니다.

콘서트 티켓팅 예약 시스템

대기열 진입부터 좌석 예약, 결제까지 이어지는 콘서트 티켓팅 서비스를 백엔드 중심으로 구현한 프로젝트입니다.

동시 요청 조건을 통합 테스트로 재현하고, 좌석·예약·결제의 책임과 경계를 나눠 중복 처리와 상태 불일치를 막았습니다.



DATE
2025.11 – 2026.05

TEAM
1인 개발

ROLE
백엔드

API 서버(Spring Boot)
콘서트 조회·예약·결제·대기열 처리

MySQL
예약·결제·포인트·좌석 데이터 저장

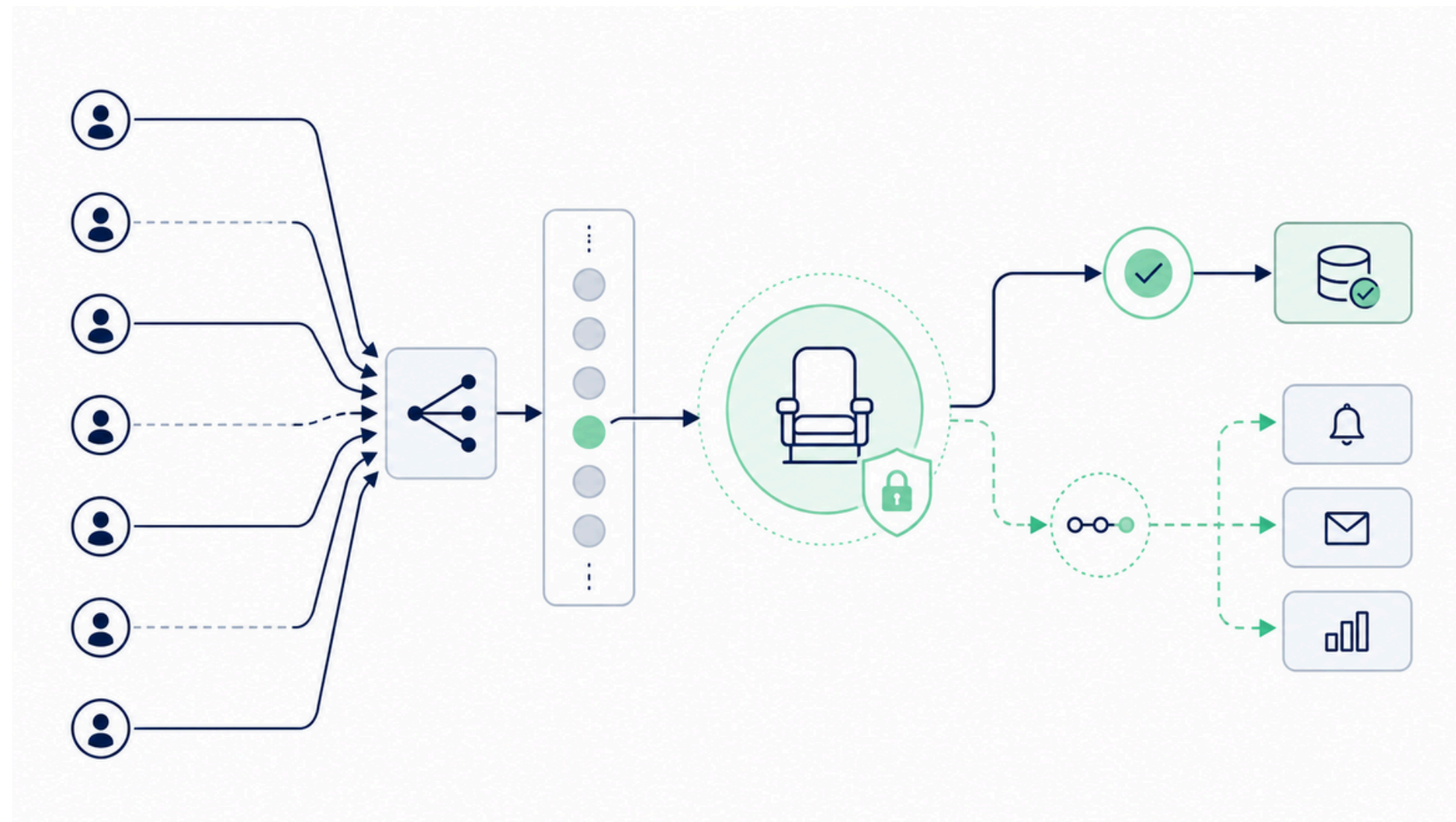
Redis
대기열·좌석 조회 캐시·분산락 처리

Kafka
예약 확정 이벤트 발행·데이터 플랫폼 연동

좌석 요청의 경쟁을 제어하고 확정 상태를 분리한 흐름

중복 예약을 막고, 확정된 상태만 남기는 것을 목표로 했습니다.

좌석 ID 기준으로 경쟁을 직렬화하고, 예약·결제·포인트는 원자적으로 확정했습니다.
커밋 이후 알림·통계 같은 후속 처리는 Kafka 이벤트로 분리했습니다.



같은 좌석의 중복 예약을 막는 좌석 기준 경쟁 제어

CONTEXT 같은 좌석에 요청이 동시에 들어오면,
둘 다 빈 좌석으로 판단해 임시 예약이 중복될 수 있었습니다.

DECISION 좌석 ID를 경쟁 자원으로 정의하고,
Redis 락·소유권 토큰·만료 시간을 함께 관리했습니다.
DB 트랜잭션과 락 해제 시점을 분리해 커밋 경계를 명확히 했
습니다.

PROOF 10개 동시 요청에서 TEMP_HOLD 성공 1건,
활성 예약 1건만 남는지 검증했습니다.
예약·결제 3개 동시 요청에서도 완료 예약·확정 좌석 각 1건을
확인했습니다.

검증한 장면
재현 조건과 최종 상태를 함께 확인했습니다.

같은 좌석 10회 동시 요청
TEMP_HOLD 성공 1건
활성 예약 1건

예약·결제 3회 동시 요청
완료 예약 1건
확정 좌석 1건

재요청에도 결제·예약 상태를 한 번만 반영하는 멍등 처리

CONTEXT

같은 예약에 결제 요청이 반복되면
결제·포인트 차감·예약 확정이 중복 생성될 수 있었습니
다.

DECISION



reservationId로 기존 결제를 먼저 확인하고,
조건이 다르면 거부했습니다.
신규 결제는 포인트·예약·결제를 한 트랜잭션으로 묶고 유
니크 제약을 더했습니다.

PROOF

같은 요청 2회와 동시 요청 5개에서
결제·포인트 차감·확정 예약이 각 1건만 남는지 검증했습니
다.

검증한 장면

재시도와 동시 요청을 같은 자원 기준으로 확인했습니다.

같은 결제 요청	2회
결제	1건
포인트 차감	1건

동시 결제 요청	5개
결제	1건
확정 예약	1건
포인트 차감	1건

동시성 문제를 해결한 핵심 설계 근거

DECISION 01

경계 분리
외부 검증·DB 저장·후속 처리를 나눠 실패 범위를 좁혔습니다.

DECISION 02

역등 기준
reservationId와 좌석 ID로 중복 여부를 판단했습니다.

DECISION 03

DB 최종 방어선
애플리케이션 검증 뒤에도 유니크 제약으로 동시 요청을 한 번 더 막
았습니다.

DECISION 04

커밋 이후 후속 처리
핵심 상태를 커밋한 뒤 후속 처리를 이벤트로 분리했습니다.

[GitHub 레포 ↗](#)

THANK YOU

문제를 재현하고 원인을 좁혀
구조적으로 해결하는 개발자입니다

맛잇온에서 동시성.트랜잭션 경계와 테스트 실행 구조를 개선했고
콘서트 티켓팅에서 좌석.결제 정합성 문제를 재현하고 해결했습니다.